Measurement Systems and Laboratory

Thermistor Calibration and Arduino Temperature Measurement

Laboratory Session Six

Group 15

Pouya Asghari

Amir Mehdi Golchin

School of Mechanical Engineering

College of Engineering, University of Tehran

First semester of academic year 1404–1405 (Iranian calendar)

About this translation

This edition translates the supplied Persian report into English and provides editable LaTeX. It retains the calibration data, reported fit, equations, figures, and code listings. Student numbers are omitted. The source's second author name is transliterated as Amir Mehdi Golchin; the preferred Latin spelling should be confirmed.

Original numerical and circuit statements are retained with clearly separated editorial notes. In particular, the calibration resistance formula has a sign error and the fourth LM324 output pin is misstated. The source code is archived unchanged; it has not been uploaded to hardware or executed in MATLAB during preparation.

Contents

1 Temperature Measurement Experiment **2**
1.1 Calibration . . . 2
1.2 Implementation of the temperature measurement system . . . 5
1.3 Code explanation . . . 9

A Original Code Listings **10**
A.1 Arduino calibration sketch . . . 10
A.2 MATLAB calibration and plotting . . . 10
A.3 Arduino temperature measurement sketch . . . 11

B Original References **12**

C Editorial Review and Reproduction Notes **12**
C.1 Calibration equation sign . . . 12
C.2 LM324 pinout and circuit interpretation . . . 12
C.3 Reference temperature and ADC resolution . . . 12
C.4 Software and archive status . . . 12
C.5 Official documentation for review . . . 13

1 Temperature Measurement Experiment

The objective of this assignment is to build and calibrate a temperature measurement system using a thermistor. A thermistor is a resistor whose resistance changes with temperature. NTC and PTC thermistors have negative and positive temperature coefficients, respectively: their resistance decreases or increases as temperature rises. The thermistor used in this experiment is an NTC device, shown below.



Figure 1: Waterproof NTC thermistor.

1.1 Calibration

This section presents the calculations and measurements used to calibrate the thermistor and determine its resistance–temperature relationship. The model is

$$R = R_0 \exp\left[B\left(\frac{1}{T} - \frac{1}{T_0}\right)\right].$$

Here, T_0 and R_0 are the reference temperature and resistance at room temperature (25°C), expressed in kelvin and ohms. Calibration is required to determine B and R_0. A thermometer and a water-filled beaker are used to record temperature alongside the thermistor resistance obtained from the Arduino calibration circuit and program. Measurements are taken at seven stages while gradually increasing the water temperature. The setup is shown below.

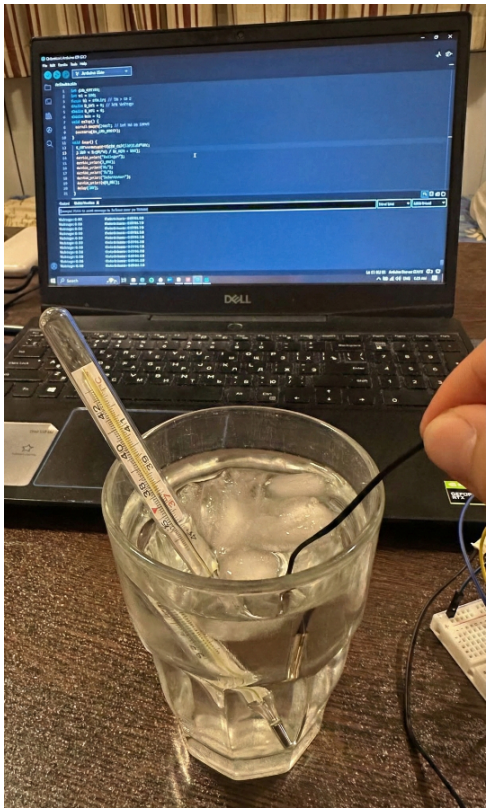


Figure 2: Experimental calibration setup.

The corresponding temperature and resistance measurements are listed in Table 1.

Table 1: Measured calibration data.

Temperature (°C)	Resistance (Ω)
3	28034
12	18419
25	9637
33	6998
42	4915
55	3117
70	1890

The MATLAB Curve Fitter tool is used to fit the exponential thermistor model to the data in Table 1 and determine B and R_0. The source reports the following fitted coefficients and goodness of fit.

Fitted constants

$B = 3891.5751$ K.

$R_0 = 9953.4133$ Ω.

Coefficient of determination: $R^2 = 0.9995$.

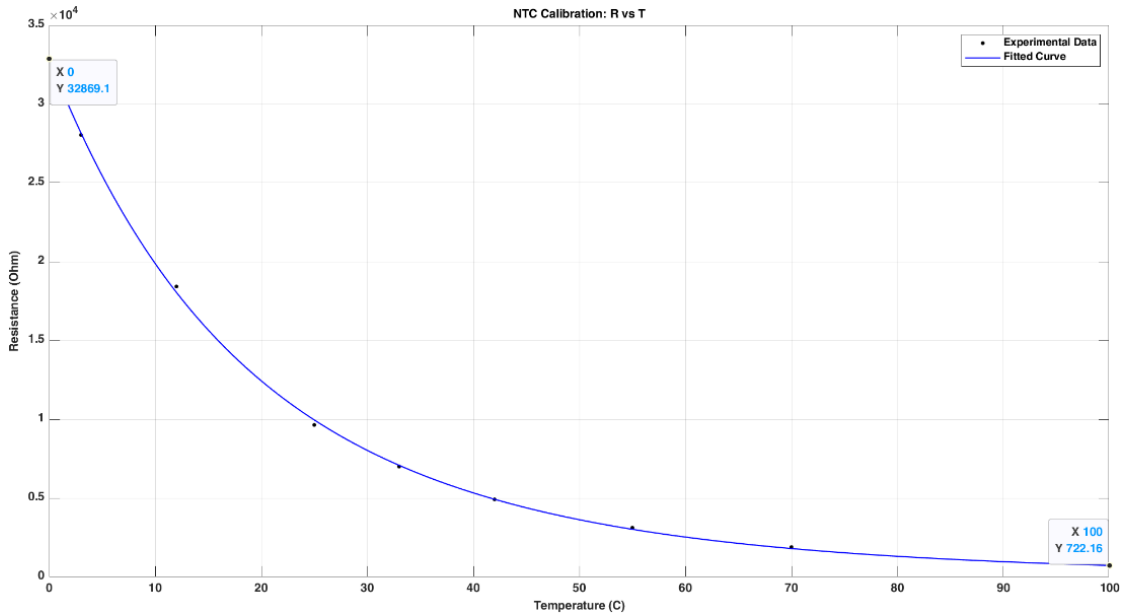


Figure 3: Thermistor calibration curve.

The rounded coefficients adopted are $B = 3892$ K and $R_0 = 9953\ \Omega$. The nominal reference resistance is expected to be 10000 Ω. The fitted value is considered acceptable within the thermistor's stated 5% tolerance.

The calibration circuit is a voltage divider containing a 10 kΩ fixed resistor and the thermistor. The midpoint voltage is read at A1. The source gives the following equations for recovering thermistor resistance.

$$V_{\text{out}} = V_{A1} = \frac{R_{\text{NTC}}}{R_{\text{NTC}} + R} V_{\text{in}} = \frac{R_{\text{NTC}}}{R_{\text{NTC}} + 10^4} (5\ \text{V}).$$

$$R_{\text{NTC}} = \frac{V_{\text{out}} 10^4}{V_{\text{out}} - 5}, \qquad V_{\text{out}} = \frac{N_{A1}}{1023} (5\ \text{V}).$$

The Arduino calibration program performs these calculations at each temperature and prints the corresponding resistance to the Serial Monitor. The voltage-divider circuit is shown below. The sign of the source's resistance formula is reviewed in the editorial appendix.

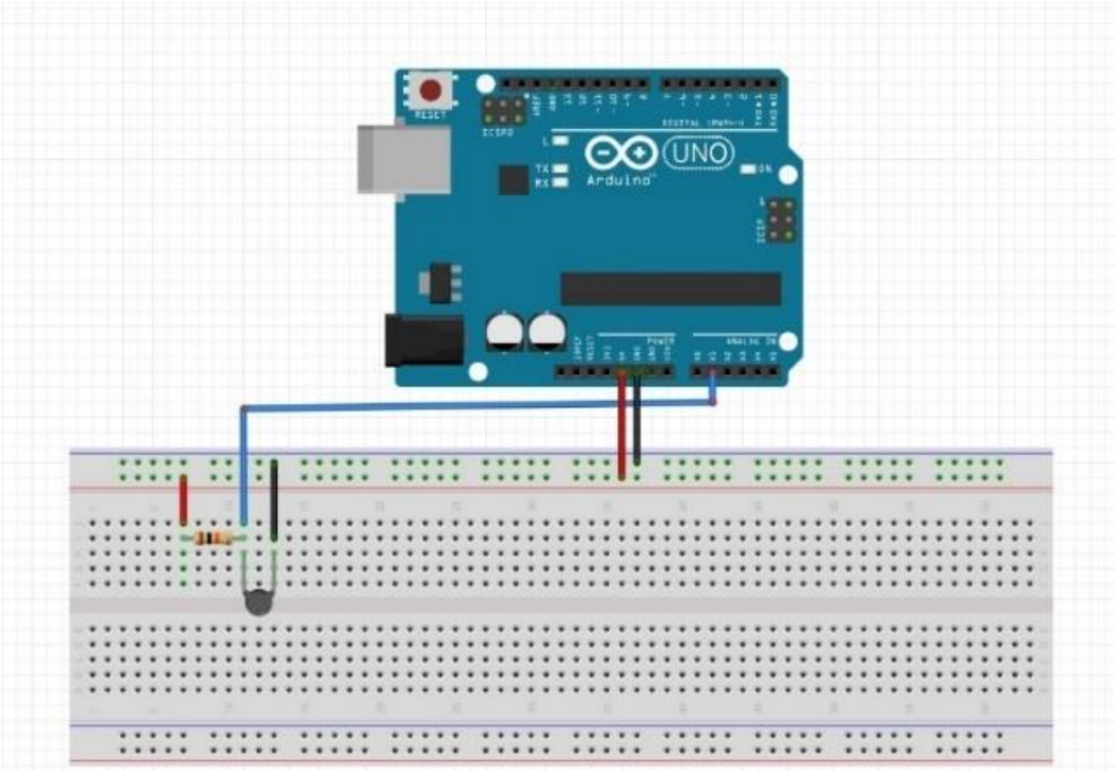


Figure 4: Voltage divider used for calibration.

1.2 Implementation of the temperature measurement system

The first design step is to calculate the parameters intended to improve the measurement resolution. The thermistor voltage at 0°C is obtained from the fitted resistance model and voltage-divider relation. This voltage is used as an offset so that the measurement range starts near zero volts at 0°C.

$$R(0^\circ\mathrm{C}) = 32869.1\ \Omega, \qquad V_0 = \frac{32869.1}{32869.1 + 10^4}(5\ \mathrm{V}) \approx 3.83\ \mathrm{V}.$$

A differential operational-amplifier circuit is used to subtract the two signals. Its voltage–temperature characteristic is shown below.

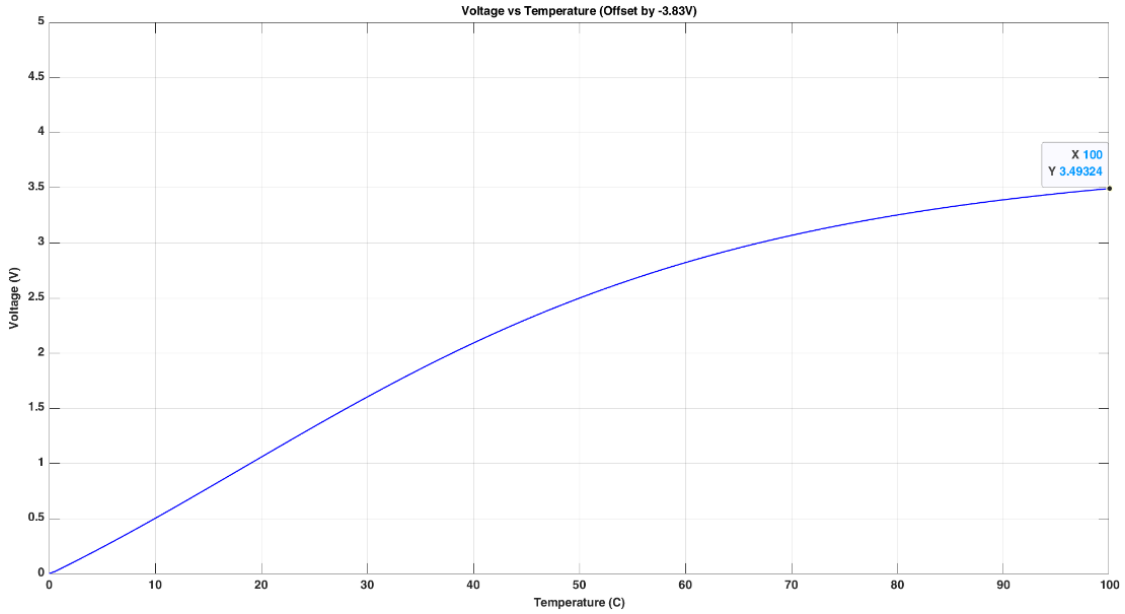


Figure 5: Voltage versus temperature after the proposed differential offset.

The source proposes reducing the ADC upper reference from 5 V to 3.49 V, distributing the available ADC codes across a narrower voltage range to improve resolution. In addition to the differential circuit, the Arduino AREF voltage is adjusted using a potentiometer.

The implementation uses the high-impedance differential-amplifier circuit presented in the LM324 datasheet.

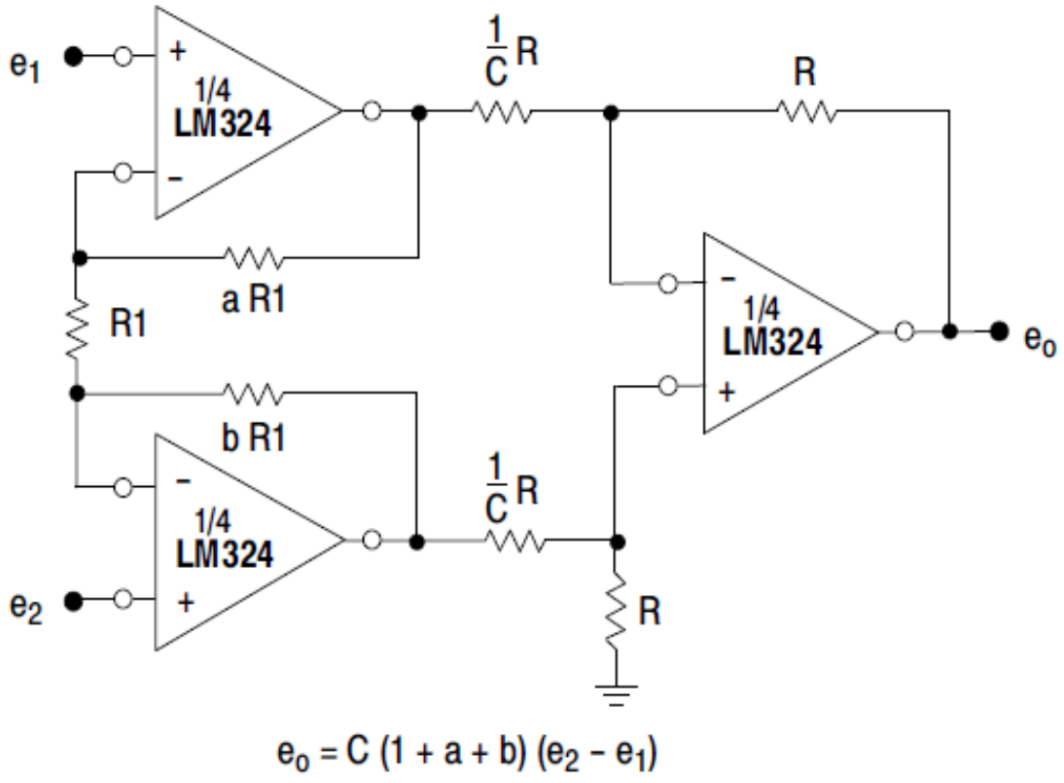


Figure 6: High-impedance differential-amplifier topology.

According to the circuit's expression, setting $a = b = 0$ and $1/C = 1$ yields only the difference between input signals e_1 and e_2, without additional gain. This arrangement is used to reduce loading and address impedance matching. The source describes a simple differential circuit with larger resistors, such as 100 kΩ or 1 MΩ, as an alternative to the 10 kΩ implementation.

Operational-amplifier sections 1, 2, and 4 of the LM324 are used. The pinout and the circuit based on this configuration are shown below. The original text cross-references Figure 2 here; the relevant amplifier topology is the high-impedance circuit reproduced above.

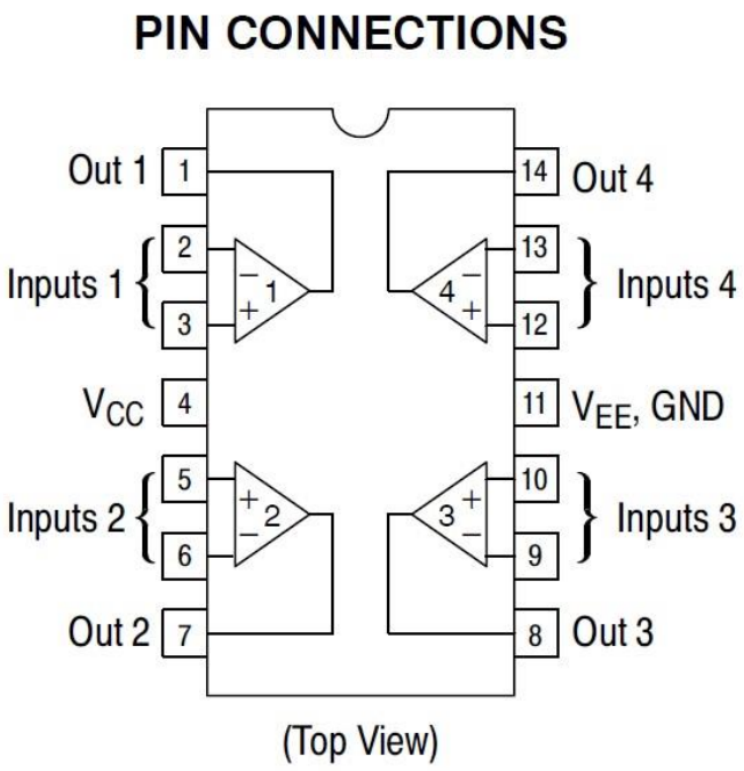


Figure 7: LM324 pin connections.

The circuit is first drawn by hand.

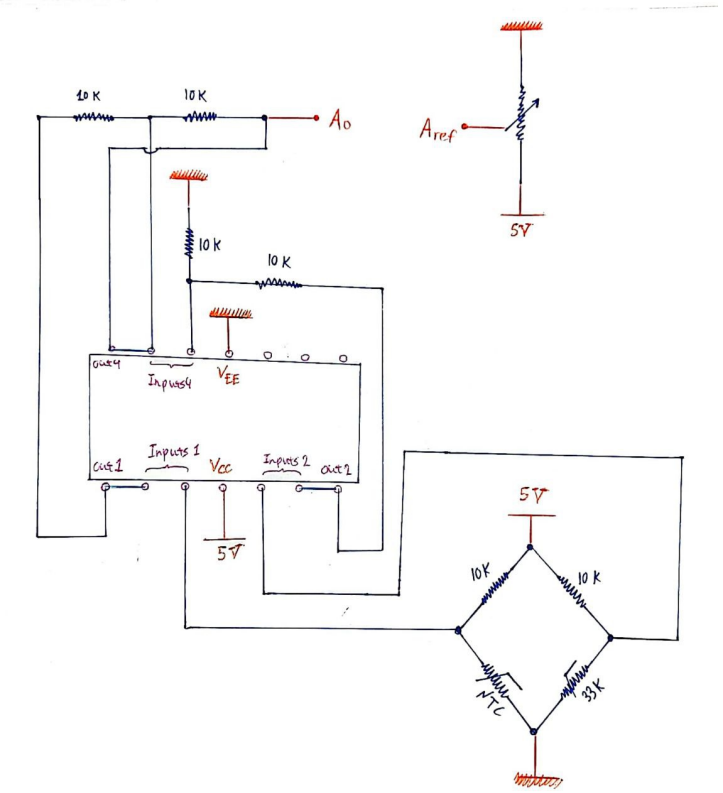


Figure 8: Hand-drawn differential-amplifier circuit.

A Fritzing circuit representation is then prepared. The original source describes this as a simulation; no native Fritzing project or simulated electrical dataset is included in the archive.

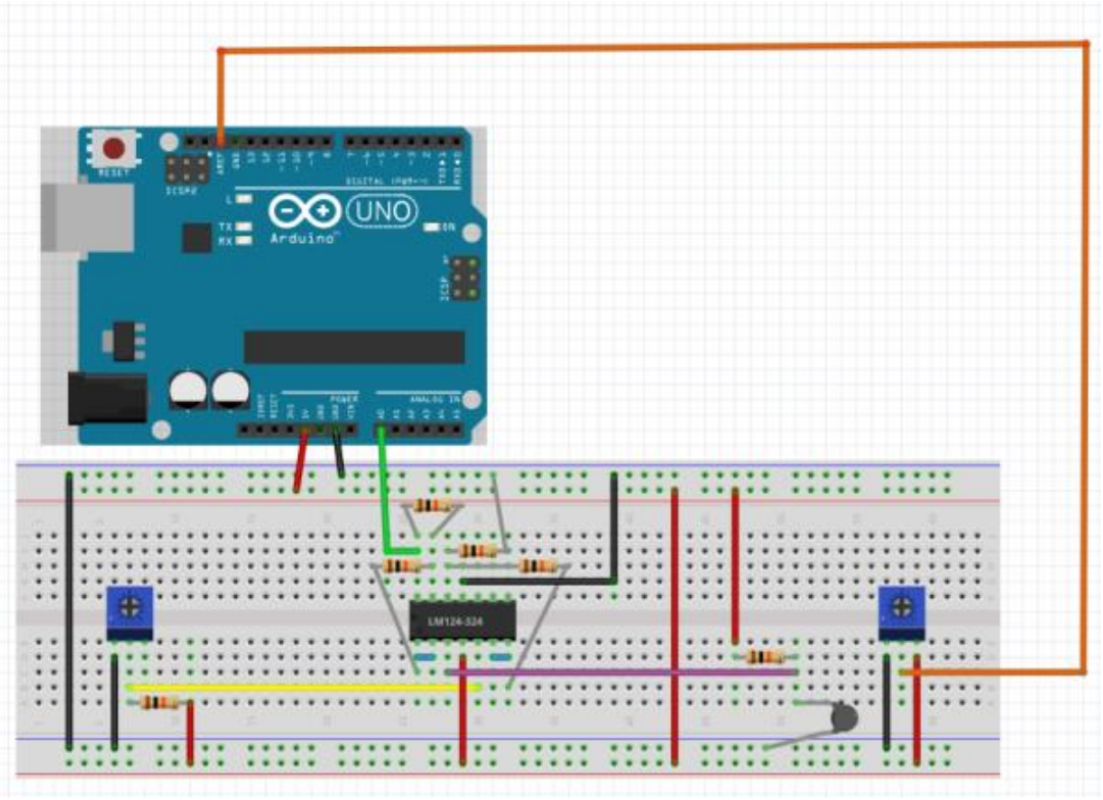


Figure 9: Fritzing representation of the circuit.

As the thermistor temperature changes, its resistance changes, and the voltage divider converts that resistance to a voltage. This voltage enters the first buffer at pin 3. A second divider, comprising a 10 kΩ resistor and a 50 kΩ potentiometer adjusted to approximately 32869 Ω, generates the fitted 0°C reference of 3.83 V and feeds the second buffer at pin 5. These signals are applied to the inverting and noninverting inputs of the fourth LM324 section at pins 13 and 12. The source states that the difference is read from pin 1 using Arduino A0; the correct fourth-section output pin is discussed in the editorial appendix. All fixed resistors in the implemented amplifier circuit are 10 kΩ. A 10 kΩ potentiometer sets the Arduino AREF voltage. Its nominal divider setting is calculated below.

$$\frac{R}{10^4}(5\text{ V}) = 3.49\text{ V} \quad \Longrightarrow \quad R = 6980\ \Omega.$$

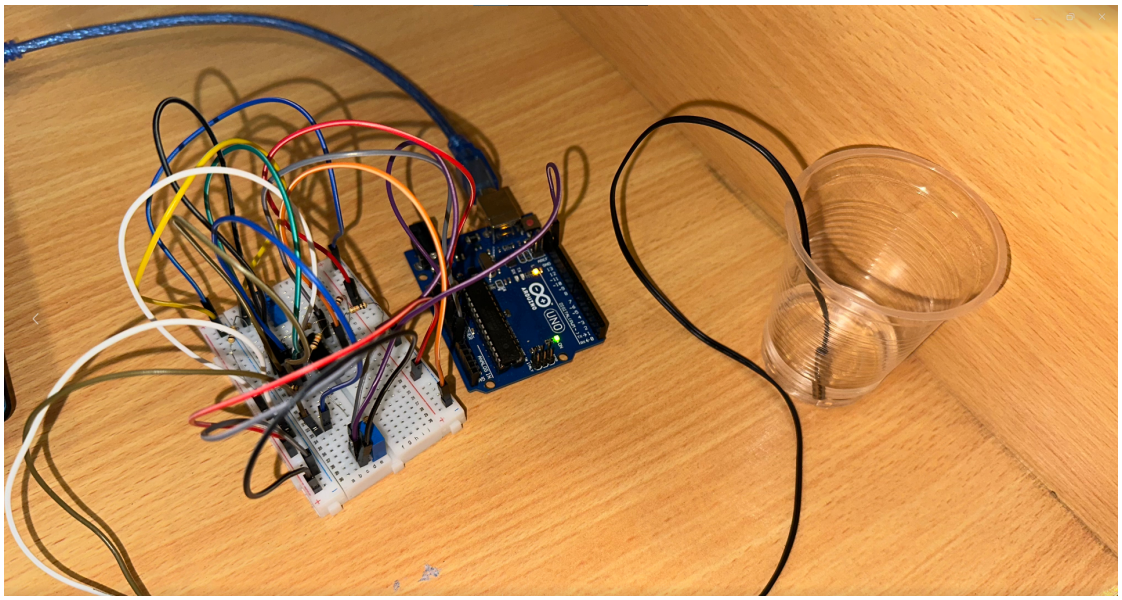


Figure 10: Assembled temperature measurement circuit.

1.3 Code explanation

The MATLAB code fits the thermistor resistance model to the calibration data using Curve Fitter functionality. The source describes the fitted relationship as a function named CalCurve. It then calculates reference resistances for the circuit, called a Watson bridge in the source, and the ADC reference adjustment, using variables described as R0 and Rref. Calibration and original/offset voltage–temperature curves are plotted. The supplied Live Script implements the fit directly; it does not contain a separate CalCurve function or an Rref calculation.

In the calibration sketch, the voltage divider is read at A1 and mapped to a voltage between 0 and 5 V, stored as V_NTC. The thermistor resistance is calculated and stored as R_NTC, then voltage and resistance are printed to the Serial Monitor. Although the original text calls A1 a digital pin, it is used here as an analog ADC input.

In the temperature measurement sketch, the differential-amplifier output is read at A0 at a nominal sampling frequency of 5 Hz using the Sampling interrupt callback. The voltage is stored as V_NTC. The resistance is recovered from the differential output and the 3.83 V offset, then converted to temperature using the fitted thermistor relationship. Voltage, resistance, and temperature are printed to the Serial Monitor.

A Original Code Listings

The native Arduino files are included unchanged. The MATLAB text file is extracted from the supplied Live Script and reproduces its executable text.

A.1 Arduino calibration sketch

```
int pin_out=A1;
int R1 = 1e4;
float T0 = 298.15; // T0 = 25 C
double V_NTC = 0; // NTC Voltage
double R_NTC = 0;
double Vcc = 5;
void setup() {
  Serial.begin(9600); // Set A0 as INPUT
  pinMode(pin_out,INPUT);
}
void loop() {
  V_NTC=analogRead(pin_out)/1023.00*Vcc;
  R_NTC = V_NTC*R1 / (V_NTC - Vcc);
  Serial.print("Voltage:");
  Serial.print(V_NTC);
  Serial.print("\t");
  Serial.print("\t");
  Serial.print("Resistance:");
  Serial.println(R_NTC);
  delay(200);
}
```

A.2 MATLAB calibration and plotting

```
clc
clear
close all;

T = [3, 12, 25, 33, 42, 55, 70]';      % Temperature (Celsius)
R = [28034, 18419, 9637, 6998, 4915, 3117, 1890]'; % Resistance (Ohms)

% The equation is: R = R0 * exp( B * (1/(273.15+x) - 1/(273.15+25)) )
ft = fittype('R0 * exp(B * (1./(273.15 + x) - 1./(273.15 + 25)))', ...
             'independent', 'x', 'dependent', 'y');

opts = fitoptions(ft);
% Updated R0 guess to 10000 to match the Ohms scale of the data
opts.StartPoint = [3500, 10000]; % [B, R0]

[fitresult, gof] = fit(T, R, ft, opts);

vals = coeffvalues(fitresult);
names = coeffnames(fitresult);

for i = 1:length(names)
    if strcmp(names{i}, 'B')
        B_val = vals(i);
    elseif strcmp(names{i}, 'R0')
        R0_val = vals(i);
    end
end

fprintf('---------------------------------\n');
fprintf('Fitted Constants:\n');
fprintf('B  = %.4f\n', B_val);
fprintf('R0 = %.4f\n', R0_val);
fprintf('Goodness of fit (R-squared): %.4f\n', gof.rsquare);
fprintf('---------------------------------\n');

T_plot = 0:0.5:100;                % Temperature range 0-100 C
```

```
R_model = feval(fitresult, T_plot); % Calculate Resistance from model

% Voltage Equation: V = R / (R + 10000) * 5
V_model = (R_model ./ (R_model + 1e4)) * 5;

% Apply Offset: Shift so 0 deg C is approx 0V (subtract 3.83)
V_offset = abs(V_model - 3.83);


figure(1);
plot(T, R, 'k.', 'MarkerSize', 10, 'LineWidth', 1); hold on;
plot(T_plot, R_model, 'b-', 'LineWidth', 1);
grid on;
xlabel('Temperature (C)', 'FontSize', 10);
ylabel('Resistance (Ohm)', 'FontSize', 10);
title(sprintf('NTC Calibration: R vs T'), 'FontSize', 10);
legend('Experimental Data', 'Fitted Curve', 'Location', 'NorthEast');

figure(2);
plot(T_plot, V_offset, 'b-', 'LineWidth', 1);
grid on;
xlabel('Temperature (C)', 'FontSize', 10);
ylabel('Voltage (V)', 'FontSize', 10);
title('Voltage vs Temperature (Offset by -3.83V)', 'FontSize', 10);
ylim([0 5]);
```

A.3 Arduino temperature measurement sketch

```
# include<TimerOne.h>
int pin_out=A0;
int R1 = 1e4;
float T0 = 298.15; // T0 = 25 C
int Sampling_Frequency = 5;
double V_NTC = 0; // NTC Voltage
double R_NTC = 0;
double Temperature = 0;
double Vcc = 5;
//B,R0 : coefficients of the calibration curve
float B=3892,R0=9953;
void setup() {
  Serial.begin(9600);
  // Set A0 as INPUT
  pinMode(pin_out,INPUT);
  analogReference(EXTERNAL);
  Timer1.initialize(1000000/Sampling_Frequency);
  Timer1.attachInterrupt(Sampling);
}
void loop() {
  //Resistance : NTC_Resistnce, V2=3.83v
  R_NTC = R1*(3.83 - V_NTC)/(Vcc - 3.83 + V_NTC);
  //Temperature Calculation
  Temperature = 1/((1/T0)+(1/B)*log(R_NTC/R0))-273.15;
  // Print Output
  Serial.print("Voltage:");
  Serial.print(V_NTC);
  Serial.print("\t");
  Serial.print("Temperature:");
  Serial.print(Temperature,5);
  Serial.print(" C");
  Serial.print("\t");
  Serial.print("Resistance:");
  Serial.println(R_NTC);
  delay(200);
}
// Sampling Function
void Sampling(){
  // ADC Range of 0 - 3.49
  V_NTC = analogRead(pin_out)/1023.00*3.49;
}
```

B Original References

1. Measurement Systems Laboratory, Session Six instructions, School of Mechanical Engineering, University of Tehran. The instructions were not supplied as a separate file.

2. `https://www.electronics-tutorials.ws/filter/filter2.html`. URL retained from the source reference list; the source does not provide a full citation.

C Editorial Review and Reproduction Notes

C.1 Calibration equation sign

For the stated divider relation $V = V_{cc}R/(R + R_1)$, rearrangement gives

$$R = R_1 \frac{V}{V_{cc} - V}.$$

The source report and calibration sketch instead use $V - V_{cc}$ in the denominator. This produces negative resistance for $0 < V < V_{cc}$. The reported positive calibration table and fitted constants should therefore be reconciled with the actual acquisition procedure. The original sketch is preserved; no hardware-tested corrected version is claimed.

C.2 LM324 pinout and circuit interpretation

For the depicted 14-pin LM324, amplifier four uses pins 12 and 13 as inputs and pin 14 as output. Pin 1 is the output of amplifier one. Verify the physical wiring against the package datasheet before using the source narrative as a wiring guide. The report's naming of a Watson bridge is retained as a source statement; the documented divider and buffered subtraction circuit should be identified from its actual connections rather than inferred to be a standard Wheatstone bridge.

C.3 Reference temperature and ADC resolution

The fitted R_0 is the resistance at 25°C. The later quantity near 32869 Ω is the predicted resistance at 0°C, not a second fitted value of the same reference resistance. Lowering ADC reference voltage reduces voltage per code; it does not by itself establish better absolute temperature accuracy. Reference stability, resistor tolerance, self-heating, amplifier offset, noise, and ADC behavior also matter.

The sketch assumes a 10-bit ADC and 3.49 V external reference. Check board support and Arduino's external-reference instructions before connecting AREF. The sampling interval is 200 ms, but loop printing is not synchronized to the interrupt. The shared V_NTC variable is not declared volatile and multi-byte reads may need an atomic snapshot on 8-bit targets. These are review items, not changes made to the original code.

C.4 Software and archive status

The MATLAB fit uses `fit`, `fittype`, and `fitoptions`, requiring Curve Fitting Toolbox. The measurement sketch depends on `TimerOne.h`; choose a TimerOne version compatible with the actual board. No native Fritzing design file is included. The source archive also contains a demonstration video of about 48.5 MiB; it is not required for LaTeX and is omitted from the browser-upload repository package. It remains in the original archive.

No hardware accuracy test, Arduino compilation, or MATLAB execution was performed during conversion. The reported R^2 is a fit statistic, not an independent temperature-accuracy measurement. Additional separately stored code should be added with its confirmed dependencies and purpose.

C.5 Official documentation for review

1. Texas Instruments, LM324 family pinout and electrical specifications: `https://www.ti.com/lit/gpn/LM124-N`.
2. Arduino analog reference documentation: `https://docs.arduino.cc/language-reference/en/functions/analog-io/analogReference/`.